

This course has already ended.

« Project starts! (https://plus.cs.aalto.fi/studio_2/k202...

Chapter 17.2: Time for the plan » (<https://plus.cs.aalt...>

CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 17 (https://plus.cs.aalto.fi/studio_2/k2022/w17/)
/ Chapter 17.1: Version Control

Chapter 17.1: Version Control

About this page:

Key questions How to maintain different versions of a software?
How do many programmers work on the same program package?

Topics Using version control systems.

What will I do? Try participating in a project using a version control system.

Indicative difficulty level: You learn a new working practice and related tools, but no new code is written in this section.

Rough estimate of workload:? 1 hour

Point value: S100

Related projects: Downloaded via Git

Introduction

You may have noticed that when working with different texts, images, or publications, that it is often useful to keep previous copies in addition to the latest version. The previous version is useful when, for example, the latest version is broken in some way or you have noticed that the latest idea was not as brilliant as you initially thought. In some jobs, it is important to document the development, i.e. how the outcome was derived. Even more often, you have to present different versions for different reviewers.

For the above reasons, working without a versioning system might result in folders which look like the following:

```
May 15 17:13 Auction.scala_final
Jul 14 23:10 Auction.scala_fix
Sep 12 09:44 Auction.scala_old_OLD
May 15 20:24 Auction.scala
Jul 14 21:20 Guide_final.doc
Oct 14 22:44 Guide_v1_final_FINAL.doc
Nov 15 16:10 Guide_OCTOBER_DRAFT.doc
```

In the example above, it looks like the user has tried to implement an own version control system by saving versions with different names. If we want to access the right file from someone else's computer, without more knowledge of the changes, this method is very inconvenient.

In programming, the differences between versions of a file are often small, but they are critical to the functionality of the program. Visual comparison of files is therefore vitally important but often particularly difficult. There are, of course, tools to support comparison like the Unix command `diff` and various graphical tools. However comparing several file versions with easily becomes quite clumsy.

It is rather common for many programmers work on the same project simultaneously. This doesn't mean they share an interactive view of the same file like in Google Docs. Each programmer needs to be able make changes to their own files and to test the whole program without interfering with each other.

At times it is important to revert unsuccessful changes or retrieve a semi-stable version of the entire program from yesterday for testing. It is not surprising that programmers have developed special version control systems (VCS) for this purpose.

Subversion

Anyone interested can listen a podcast about the birth and development of Subversion-version control system (37 min)
<https://corecursive.com/software-that-doesnt-suck-with-jim-blandy/> 
(<https://corecursive.com/software-that-doesnt-suck-with-jim-blandy/>)

Version Control Software

Version control systems save information on files and specifically the lines of code that have been changed. In addition to this, they save the time of the change, the user who made the change, and the user's description of the purpose of the changes. The version control system can be used to retrieve any saved version and copy it into the user's *working directory* as the **working copy**, i.e., as normally accessible, visible file(s). Version control software can also compare different versions and show differences between them. If two versions do not have conflicting changes, they can be merged into a new version.

Since a project or file storage (*repository*) in a version control system is often used by multiple users on multiple devices, it is common to create a version control repository on some server. Such a version control server usually has a backup system so that the source code and

development history of the software project won't be lost. Developers often use version control as a publishing tool, by naming/numbering published versions and automating their installation or packaging for users.

In this course, we use the version control software called **Git**, which is one of the most popular choices in the industry as well as in hobby projects. Since the concepts and commands used by different version control systems are quite similar, after this chapter you can easily learn other version control systems.

Git Version Control System

Git is a distributed version control system (DVCS) created by Linus Torvalds for the needs of the *Linux* operating system project in 2005. On Git's own site <https://git-scm.com>  (<https://git-scm.com/>), you will find the *Pro Git guidebook* in English and other learning resources, including tutorial videos. If you don't have git installed yet, the site provides downloads for easy-to-use installers.

IntelliJ comes with tools that interact with (and depend on) the Git tool installed on your system. You can check if everything is already working by checking the menu **Preferences -> Version Control -> Git** and pressing the button **Test**. IntelliJ should show you the Git version number below.

If you don't have it installed yet, begin by installing Git. Once everything is in place IntelliJ should find the git installation automatically.

Git exercise

Git commands can be given from the command line or through graphical user interface programs. In this chapter, we practice using Git through the IntelliJ development environment. (If prefer a different IDE or want to use the command line, you can still do this exercise, by using any Git compatible tool such as the command line.).

Command line working with Git is discussed at the lecture and lecture resources are in MyCourses.

1. A working copy of the project (*Clone*)

In this exercise, you will join the development of a small unfinished program. This is done by adding a new project to IntelliJ from the online version control remote repository. Please, follow these detailed instructions:

- In IntelliJ, choose *File > New Project > From Version Control*.
- *Repository URL* is already selected on the left.
- Get from below (exercise 0) your personal remote repository address.
- Fill in the field *URL* based on the exercise information and choose a *Directory* where the repository will be placed.

- When you press clone you will be asked for *User* and *Password*. If you want to, you can IntelliJ to *Remember* them. Finally, press *Log In*.
- When prompted, select to open the project in a *New Window*

This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 0 (Tunnukset)

This course has been archived (Thursday, 1 June 2023, 12:00).

Press here to create your personal Git-repository for this chapter

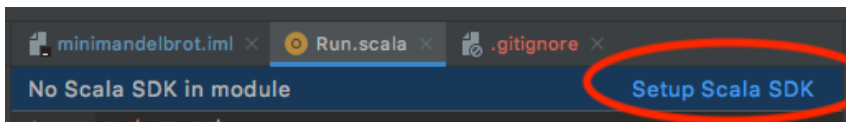
Show anyway

Note that if run you run it again you will get a new password.

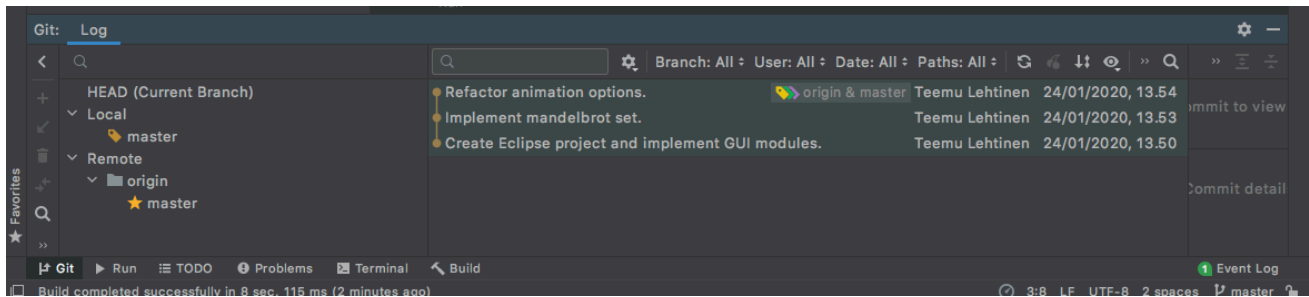
The project now appears in the IntelliJ like a normal project. Note that in the lower righthand corner it says *master* and in the lower lefthand corner there is the menu *Git*. These both indicate that the project is in version control. Take a superficial look at the latest version and files in your *working directory*. Also, try running the program from `src/mmb/Run.scala`.

Setup Scala SDK

The Scala SDK is not included with the project, but IntelliJ should prompt for it to be added. Just click the "Setup Scala SDK" on top of the edit window. If Scala 2.13 is on the list, select it. Otherwise click "Create" -> Select 2.13 and click "OK/Apply".



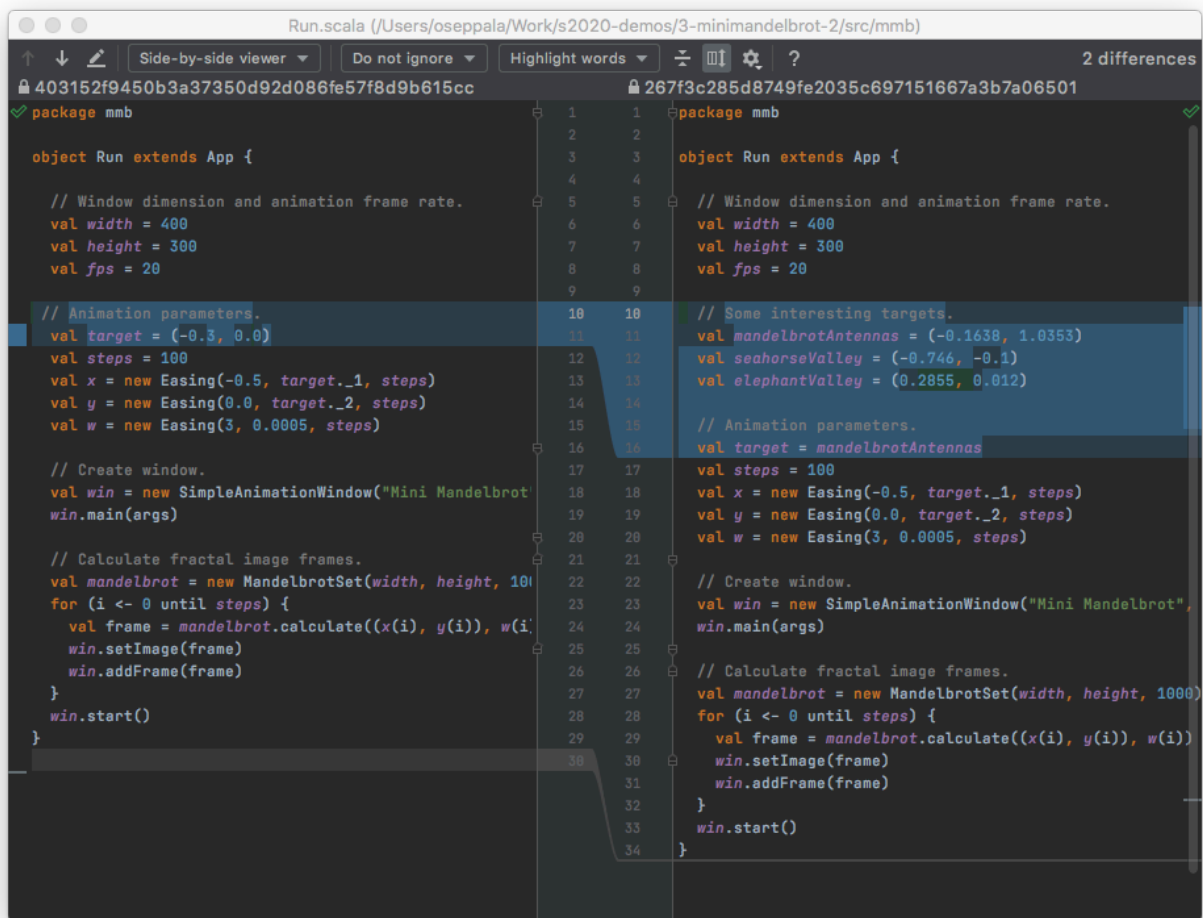
In addition to the latest version of the program, the full version history of the project was copied to your computer from the remote repository. The following figure shows a history view which can be opened by the *Git* menu in the lower lefthand corner. The program seems to have been written by another developer, Teemu. At the top, you can see the newest *HEAD* version and the additional tags indicate that both the locally stored *master* version and the *origin/master* version are at this same stage. Before the current version, Teemu has saved two previous versions.



Let's look more closely at the `Run.scala` file:

- Select the newest version in the window and double-click `Run.scala` on the left.

In the opened view, you will see the latest version on the left side and the previous version on the right. Changes between the versions have been highlighted.



 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

This course has been archived (Thursday, 1 June 2023, 12:00).

Exercise 1

Show anyway

Question 1 10 / 10

Which of the following describes *best* the last changes in *Run.scala*?

- ☐ A *for* structure has been added starting from line 28.
- ☐ *Easing* objects have been added to lines 18-20.
- ☒ **Starting from line 10 variables for coordinates have been added.**
- ☐ The title for a window has been changed on line 23.

✓ Correct!

Submit

You can now close the comparison window.

2. Submitting changes (*commit*)

We are now making a small change to the program. The Mandelbrot fractal repeats the same forms endlessly, but different coordinates of the complex plane have very different landscapes. Let's look at the "valley of seahorses":

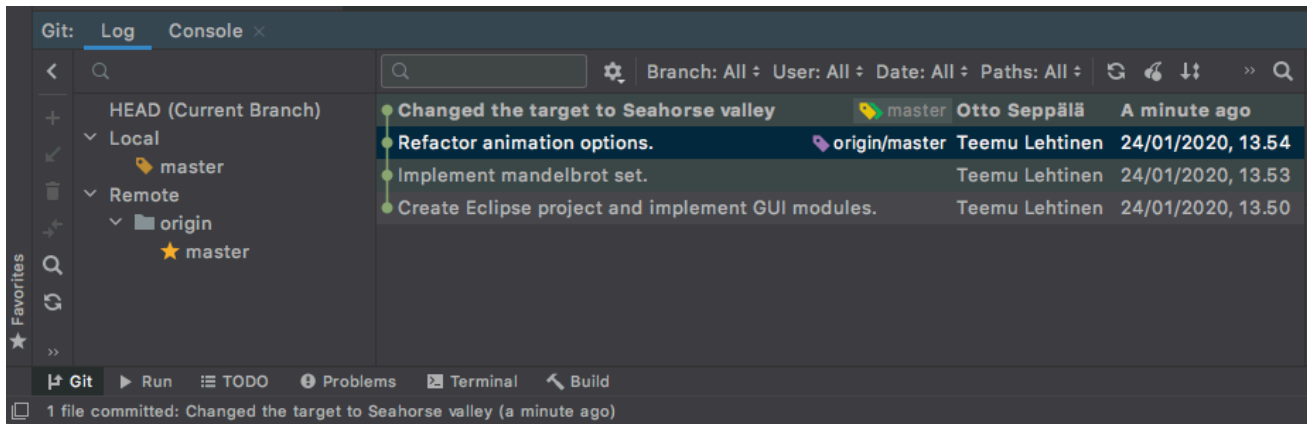
- Change line 16 of *Run.scala* to *seahorseValley*
- Save and check the program by running it.

Saving a file did not automatically make a new version of the file in the version control system! The registration of changes must be explicitly requested by the *commit* function. IntelliJ shows unregistered changes in the file by coloring it's filename blue. To maintain clarity of the version history, it is a good practice to group changes in different files related to one goal into one *commit*. It is advisable to make *commits* frequently and for one change at a time. Let us register the change we just made:

- Select *Git > Commit* from the Main menu (or click the green ok sign in the top menubar).
- On the left in the window that opens, new or changed files can be selected to be included in the commit (*Run.scala* should already be selected and in this commit we do not wish to include any other files).
- In *Commit Message* box (bottom-left corner), enter a description of the changes (for example, "Change zoom target to Seahorse Valley.") and press *Commit*.

The description is mandatory and important in practical working. The quality of the descriptions is emphasized when the size of the development team grows or when then team is placed in different locations.

3. Exporting a change to a remote repository (push)



Open the git view from the lower left again. In the previous stage, the change was registered in the **local repository**. If we look at the history now, the local repository *master* is now your version, but the remote repository *origin/master* is not matching it. As the local repository is usually private, your changes will not yet be visible to other programmers.

All changes made locally are exported separately to the *remote repository* with the **push** operation. By default, the changes are exported to the remote repository *origin*, from which your local repository was cloned in phase 1.

- Select *Git > Push* from the main menu and click *Push*.

Once you have *pushed*, the remote repository is up-to-date with local changes. Use the exercise below to check the status of the remote repository and to get points for the previous steps. (You do not need to select files from your own computer.)

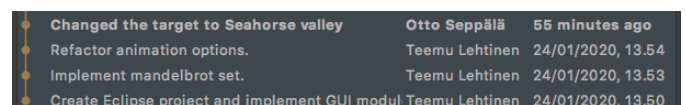
Loading assignment...

4. Applying changes from a remote repository (pull)

There is a rumor in the chatroom that after your last push Teemu has changed the program again. The latest version can be imported to the local repository and working copy using *pull* function:

- Select *Git > Pull*

Now the local repository is up to date with the changes in the remote repository. Try the revised program and note its functionality after the changes. Fractal coloring now changes with the bounding box of the view.



(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_images/git_pulled_IJ.png)

5. New file

Documentation is also needed in a successful project. Now let us add to the top directory of the project, alongside `src` (not within it) a new file called `README.md`. Use the *New > File* function. You can write anything you want in the file. We will not read or review its content in this exercise, so you can leave it empty.

The new file should now appear in the project view above the "swing-jar file" and it should be colored orange. This means that the file is not under version control. IntelliJ will ask us if we want to add the file to Git. Press Add. Notice that the file turned green. It is however not in the local repository or the remote repository yet. Let's fix that.

- As in phase 2 do *Commit*.
- As in phase 3 do *Push*.

(Actually you could just press `commit-and-push`)

Once you have approved exporting the file, the remote repository is up-to-date with local changes. Use the exercise button below to check the status of the remote repository and to get points from the previous steps.

Go ahead

In order to simulate a variety of situations that are often encountered in real life, we will try other everyday situations with version control. The next "problem" in this task can be avoided in several different ways, but we suggest you follow the steps and the guidance in the solution. You will get valuable experience from completing this task.

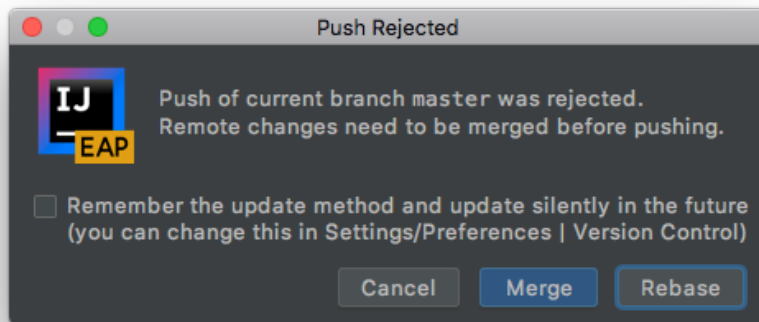
6. One more change

Let us go to the elephant valley.

- In the file `Run.scala`, on line 16, change the target to `elephantValley`.
- *Commit* as in phase 2.
- Try to *Push* as in phase 3 and look at the end result carefully.

The last step does not succeed (*Push Rejected*). According to the message, the action was rejected and the remote repository did not change (you could check this in the History view). The push was rejected because Teemu has already changed the program after your previous

`push` command. The remote repository cannot receive your export before your local repository contains all the version changes registered in the remote repository. There are a few options available for us, but for practice sake we'll select *Cancel*.



Note that this situation will not arise if, for example, you had appropriately made the *pull* function before your own *commit*. Nevertheless, in real life, one can never guess when another developer decides to save his own changes to the repository. So let's explore this very common situation:

- Do a *Pull* as in phase 4.

Unlike in the previous case, you get a notification of overlap (*Conflicts*). This time, the same row has been changed both in the versions at the remote repository and at your local repository. The version manager cannot automatically decide how the versions should be merged and therefore needs your help.

If the changes concerned different rows in the same file, the version control would merge them automatically by including both changes and your help would not be needed. However, all changes can have unpredictable consequences, so you should *always look at the files that have been modified by the pull function*.

The conflicting file is now colored orange in the list on the left. The program code in the conflicting part (*Run.scala*) looks like this.

```
<<<<<<< HEAD
  val target = elephantValley
  =====
  val target = mandelbrotAntennas
>>>>>>> 5fca0289310bf2e4153555d83267e80f0c611b5a
```

Between "<<<<<<<" and ">>>>>>>" there are conflicting lines in the two different versions. Versions are separated by the line "===== ". To resolve this conflict, you need to change this point in the code into the desired correct form by deleting the wrong line and the conflict marking entries. Only the code selected by the programmer remains. The file can be directly edited, but let us practice using a more visual tool which is helpful in dealing with possible larger conflicts:

- Select *Git > Merge > Resolve Conflicts* from the main menu.
- Press *Merge*

The view that opens is similar to the view we discussed at the beginning of the chapter that was used to compare the current version with the previous one. The left side is your version and Teemu's competing version is on the right. The middle one is the original before either of you changed it. We only want to keep Teemu's version. By clicking "<<" next to Teemu's change, you can copy the code from the competing version to the left. The middle window can be edited freely.

- If you did not do it yet, click "<<" next to Teemu's change to copy it to the result in the middle.
- Press *Apply* to close the compare view.
- Accept "Apply Changes and mark resolved"

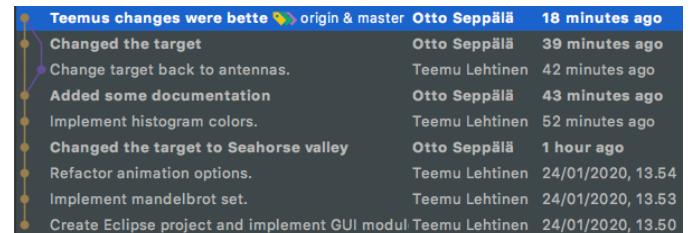
The file is now colored Blue (changed, but not yet committed) It not red, because the conflict has been fixed.

- Do a *Commit* as before to put the merged version of *Run.scala* to the repository. Write a commit message.

Conflicting versions are now *merged*. Finally, let's push the local changes to the remote repository.

- Do a *Push* as before.

Once you have approved the export, the remote repository is up to date with local changes. Use the exercise button below to check the status of the remote repository and to get points from the previous steps.



(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_images/git_merged_IJ.png)

Version control web services

In the previous exercise, a repository or a version control storage was set up for the exercise. This is also very common in real software projects. In private projects, repositories are protected, but open source project  (https://en.wikipedia.org/wiki/Open-source_software) repositories are generally accessible to anyone and can be fetched by the *Clone* function. However, exporting changes using *push* is limited to project administrators.

Network or cloud services, such as Github  (<https://github.com/>) or BitBucket  (<https://bitbucket.org/>), allow registered users to create their own projects. They are very popular in open source projects, and their use is also growing in private software projects. These services maintain a *Git repository* for the created project, which can be used in the same way as in this exercise.

Furthermore, online services allow you to view the content and history of the repository in the browser, manage user access rights, and do automatic backups. In addition, there are tools for project task management (*issue tracking*). Open source projects have adopted a new set of working practices enabled by the services. For example, if you would like to improve the Scala programming language, you can make an unofficial *fork* of the Scala-project  (<https://github.com/scala/scala>) without asking for anyone's permission. Once you have made changes to the program code, you can make a *pull request* to the official Scala programming language repository. If the change is a proper and justified improvement, it can be accepted after careful evaluation and approval and merged with the official Scala project.

Aalto GitLab

Aalto University maintains an open source *GitLab* online service that Aalto students can use for version control. The service resembles the GitHub online service, but access to it is restricted to only Aalto students and staff. [Version.aalto.fi](https://version.aalto.fi) is often preferred to other services on Aalto courses as it is connected to aalto accounts and the files are stored on Aalto Servers.

<https://version.aalto.fi>  (<https://version.aalto.fi/>)

Just a local repository

If you do not want to use network services for one reason or another, you can create an exclusively local repository. As no remote repository has been specified, the *pull* and *push* operations won't be used. However, the local repository still supports comparisons between versions etc.

To create a local repository select *VCS > Enable Version Control Integration* and select Git in the pop-up. You can add a remote repository later with *Manage remotes*.

Summary

- Version control is essential in programming projects for following, managing, and publishing changes.
- In addition to a *working copy*, a *repository* is needed where the entire development history is stored.
- In addition to saving files, changes should be frequently sent to the repository using the commands *add* and *commit*. It is good practice to work on one feature or improvement at a time, and submit each in their own, clean commit.
- Changes from the remote repository are imported with the *pull* command to the local repository, and changes in the local repository are exported to the remote repository with the *push* command.
- Resolving *conflicts*, i.e. contradictory changes, requires a programmer to step in.

Learn more about branching

In the version control history view, the *master* is the default branch of the repository. In this assignment, it was noted that the *origin / master* in the remote repository is a separate development branch that can be different from the local one. In the exercise, we merged those development branches with the *pull* command.

Additional development branches can be created at any time. For example, the development of a new function of the program can be split into a *feature-x* branch, while keeping the *master* branch for the published program maintenance. When the new function is to be released, the *feature-x* can be merged with the master development branch. Changes carried out after the split will be added, if possible, to the *master* branch. After the branches have been merged, the master contains the changes from both branches.

The *working copy* can easily be switched for another development branch, whereby the *working directory* files are replaced by the files in the latest version of the selected branch. The most important commands for managing development branches are *branch*, *checkout*, and *merge*. More information can be easily found online.

<http://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>  (<http://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>)

Try Git, Command Line Usage

If you have extra time left for this round, I recommend trying out the Git command line exercise below. If you are heading to the programming industry, you will also need to use Git from the command line. The names of the commands are already familiar after reading this chapter!

<http://try.github.com/>  (<http://try.github.com/>)

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

« Project starts! (https://plus.cs.aalto.fi/studio_2/k202...)

Chapter 17.2: Time for the plan » (<https://plus.cs.aalt...>)